```
        System.out.println("float fMax equals to " + fMax);
        System.out.println("float fLit equals to " + fLit);
        System.out.println("Size of float is " + Float.SIZE + " bits");
    }
```

Κώδικας 3.5*: Μεταβλητές τύπου float*

Ακολουθεί η έξοδος του κώδικα 3.5

```
float fMin equals to 1.4E-45
float fMax equals to 3.4028235E38
float fLit equals to 3.15
Size of float is 32 bits
```

Προσέξτε πως η σταθερά 3.15 ακολουθείται από το f, εναλλακτικά μπορεί να χρησιμοποιηθεί το F, ώστε να καθοριστεί ο τύπος της σταθεράς ως float. Χωρίς το f ή F, ο τύπος της σταθεράς είναι ο double που παρουσιάζεται αμέσως παρακάτω. Όπως βλέπουμε στην έξοδο του κώδικα 3.5, οι τιμές των fMin και fMax έχουν εκφραστεί με τον επιστημονικό συμβολισμό (scientific notation). Ο επιστημονικός συμβολισμός μπορεί να χρησιμοποιηθεί και για να εκφραστούν κυριολεκτικές σταθερές τύπου float ή double μέσα στον πηγαίο κώδικα. Επομένως, η ελάχιστη πραγματική τιμή που μπορεί να αναπαρασταθεί με τον float είναι 1.4E-45, δηλαδή 1.4x10-45 και η μέγιστη είναι 3.4028235E38, δηλαδή 3.4028235x1038, ο δε χώρος που καταλαμβάνεται στη μνήμη είναι 32 bits.

Οι μεταβλητές τύπου float δύνανται να λάβουν επιπλέον τις ειδικές τιμές Float.NEGATIVE_INFINITY, Float.POSITIVE_INFINITY και Float.NaN (Not a Number). Επιπλέον, σε αντίθεση με τους ακέραιους τύπους, οι πραγματικοί υποστηρίζουν δύο μηδέν, το +0f και το -0f. Αυτό οφείλεται στην κωδικοποίηση IEEE 754 που είναι συμμετρική μεταξύ θετικών και αρνητικών.

Για παράδειγμα, η τιμή Float.NEGATIVE_INFINITY επιστρέφεται όταν ένας αρνητικός αριθμός τύπου float διαιρείται με το +0f ή αντίστοιχα ένας θετικός float διαιρείται με το -0f. Επίσης, Float.NEGATIVE_INFINITY ή Float.POSITIVE_INFINITY μπορεί να επιστραφεί ως αποτέλεσμα έκφρασης στην οποία συμμετέχουν αυτές οι τιμές. Αν όμως έχουμε διαίρεση με Infinity, το αποτέλεσμα είναι NaN. Η τιμή Float.NaN επιστρέφεται και σε διάφορες άλλες περιπτώσεις, όπως για παράδειγμα όταν ζητηθεί η τετραγωνική ρίζα αρνητικού αριθμού.

Η διαίρεση ακεραίου με το 0 παράγει εξαίρεση[5] του τύπου “διαίρεση με το 0” (division by zero). Για παράδειγμα, ο κώδικας που ακολουθεί παράγει εξαίρεση.

```
int i = 1, j = 0;
System.out.println(i / j);
```

Αντίθετα, η διαίρεση πραγματικού με το 0 επιστρέφει Infinity. Για παράδειγμα, ο κώδικας που ακολουθεί τυπώνει Infinity.

```
float f=1, r=0;
System.out.println(f/r);
```

Αντίθετα, ο κώδικας

```
float nAN=(float)Math.sqrt(-1);
System.out.println(nAN);
```

θα τυπώσει NaN.

Σημειώστε πως η συνάρτηση sqrt της κλάσης Math επιστρέφει την τετραγωνική ρίζα του ορίσματός της. Στη συγκεκριμένη κλήση επιστρέφει την τετραγωνική ρίζα του -1. Όπως είναι γνωστό, τέτοιος πραγματικός δεν υπάρχει, ως αποτέλεσμα η sqrt επιστρέφει NaN. Επίσης, ο τύπος της τιμής που επιστρέφει η sqrt είναι

[5] Οι εξαιρέσεις είναι ο μηχανισμός διαχείρισης απροσδόκητων γεγονότων. Παρουσιάζονται αναλυτικά στην ενότητα 16. Για την ώρα θεωρήστε την εξαίρεση συνώνυμη του λάθους χρόνου εκτέλεσης.

double. Επομένως, στην κλήση που συζητάμε θα επιστρέψει NaN τύπου double (Double.NaN). Για τον λόγο αυτόν αναγκαζόμαστε να προχωρήσουμε σε μετατροπή του τύπου από double σε float προκειμένου να εκχωρήσουμε την επιστρεφόμενη τιμή στη μεταβλητή nAN τύπου float. Όπως φαίνεται στον κώδικα, για να μετατρέψουμε το αποτέλεσμα της sqrt τοποθετήσαμε πριν την κλήση της το όνομα του νέου τύπου μέσα σε παρενθέσεις. Θα πρέπει να έχουμε υπόψη μας πως μια τέτοια μετατροπή γίνεται με ευθύνη του προγραμματιστή και μπορεί να οδηγήσει σε απώλεια ακρίβειας.

Η κωδικοποίηση IEEE 754 μας δίνει τη δυνατότητα στο ίδιο μήκος μνήμης να αναπαραστήσουμε μεγαλύτερους αριθμούς από ότι η κωδικοποίηση συμπληρώματος ως προς 2 που χρησιμοποιείται για την κωδικοποίηση των ακέραιων. Ωστόσο, είναι σημαντικό να κατανοήσουμε ότι οι πραγματικοί αριθμοί αποθηκεύονται στη μνήμη προσεγγιστικά. Ως αποτέλεσμα συγκρίσεις πραγματικών με χρήση του τελεστή ισότητας (equality operator), ==, ή των υπόλοιπων σχεσιακών τελεστών (>, <, >=, <= και !=, ενότητα 4.1.4) δεν είναι αξιόπιστες και πρέπει να αποφεύγονται.
Για παράδειγμα, ο κώδικας

```
float f=0.1f+0.000000001f;
System.out.println(f==0.1f);
```

θα τυπώσει true.
Προσέξτε τον ακόλουθο κώδικα

```
float d1 = 1, d2=1;
for (int j = 1; j <= 10; j++) {
     d1 = d1+0.00001f;
}
d2=d2+0.00001f * (10);
System.out.println(d1==d2);
```

Οι μεταβλητές d1 και d2 αρχικοποιούνται και οι δύο στην τιμή 1. Σε κάθε βήμα της επαναληπτικής διαδικασίας for, προστίθεται η τιμή 0.00001f στην d1. Έχουμε ένα σύνολο 10 επαναλήψεων, άρα στην d1 προστίθεται 10 φορές το 0.00001. Στη συνέχεια προσθέτουμε στην d2 το 10*0.00001. Αναμένουμε επομένως ότι τα d1 και d2 είναι ίσα μεταξύ τους. Ωστόσο, ο κώδικας θα τυπώσει false, καθώς η τιμή της d1 έχει υπολογιστεί ως 1.0001001 και της d2 ως 1.0001. Για αυτόν τον λόγο, ανάλογα με τις ανάγκες της εφαρμογής μας, η σύγκριση μεταξύ πραγματικών αριθμών μπορεί να γίνεται προσεγγιστικά.

```
static boolean approximateEquals(float f1, float f2, float
epsilon) {
     return Math.abs(f2 - f1) < epsilon;
}
```

Χρησιμοποιήστε τη συνάρτηση approximateEquals για να συγκρίνετε αριθμούς float. Για να δείτε το αποτέλεσμά της, καλέστε την ως εξής:

```
System.out.println(approximateEquals(d1, d2, 0.0001));
```

Η τρίτη παράμετρος συνιστά μια οριακή τιμή. Αν η απόλυτη τιμή της διαφοράς των δύο float είναι μικρότερη από αυτήν την οριακή τιμή, οι αριθμοί θεωρούνται ίσοι. Αν για παράδειγμα θέλουμε να συγκρίνουμε τα μήκη δύο οδών και το μήκος της πρώτης το μετρήσουμε στα 2 km και αυτό της δεύτερης στα 2 km και 1 χιλιοστό του εκατοστού, μπορεί να τις θεωρήσουμε ισομήκεις.

Γενικότερα, ο τύπος float θα πρέπει να αποφεύγεται για διαχείριση τιμών που απαιτούν ακρίβεια. Για τέτοιου τύπου πληροφορίες συνίσταται η χρήση της κλάσης Java.math.BigDecimal. Το ίδιο ισχύει και για τον τύπο double που παρουσιάζεται αμέσως παρακάτω.

double Πρόκειται για τύπο αποθήκευσης πραγματικών αριθμών, επίσης κωδικοποιημένο με το IEEE 754. Η διαφορά του από τον float είναι πως ο double έχει μήκος 64 bits. Πρόκειται για τύπο κινητής υποδιαστολής διπλής ακρίβειας (double-precision 64-bit IEEE 754 floating point). Εξαιτίας του μεγαλύτερου μεγέθους του

μπορεί να φιλοξενήσει πραγματικές τιμές με μεγαλύτερη ακρίβεια από τον float. Στην τυπική περίπτωση είναι ο τύπος που θα πρέπει να χρησιμοποιούμε όταν θέλουμε να διαχειριστούμε πραγματικούς αριθμούς. Ο float θα πρέπει να χρησιμοποιείται μόνο στις περιπτώσεις που έχουμε πολλά δεδομένα από πραγματικούς αριθμούς, χρειάζεται να κάνουμε οικονομία στη μνήμη και είμαστε σίγουροι πως η ακρίβεια που προσφέρει, μας αρκεί. Ο τύπος double διαθέτει επίσης τιμές Double.PositiveInfinity, Double.NegativeInfinity, Double.NaN, αρνητικό και θετικό μηδέν με παρόμοια χαρακτηριστικά με τις αντίστοιχες τιμές του float.
Στον κώδικα 3.6 παρατίθεται παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου double.

```
    static void doubleDemo() { //Κώδικας 3.6
        double dMin = Double.MIN_VALUE, dMax = Double.MAX_VALUE, dLit =
3.4028235E38;
        System.out.println("double dMin equals to " + dMin);
        System.out.println("double dMax equals to " + dMax);
        System.out.println("double dLit equals to " + dLit);
        System.out.println("Size of double is " + Double.SIZE + "bits");
    }
```

Κώδικας 3.6*: Δήλωση και αρχικοποίηση μεταβλητών τύπου double*

Ακολουθεί η έξοδος του κώδικα 3.6.

```
double dMin equals to 4.9E-324
double dMax equals to 1.7976931348623157E308
double dLit equals to 3.4028235E38
Size of double is 64bits
```

Επομένως, η ελάχιστη τιμή του double είναι 4.9x10-324, η μέγιστη 1.7976931348623157x10308 και το μέγεθός του 64 bits. Παρατηρήστε πως στην αρχικοποίηση της dLit έχει χρησιμοποιηθεί επιστημονικός συμβολισμός. Οι σταθερές με δεκαδικά που δεν έχουν χαρακτηριστεί ως float θεωρούνται εξ' ορισμού πως είναι τύπου double. Μπορεί να συνοδευτεί μια τέτοια σταθερά από τον χαρακτήρα D ή d προκειμένου να δηλωθεί ο τύπος της σαφώς, ωστόσο αυτό αποφεύγεται από σύμβαση.

Παρότι ο double προσφέρει μεγαλύτερη ακρίβεια από τον float, δεν παύει να αντιπροσωπεύει προσεγγιστικά τους πραγματικούς αριθμούς και η χρήση του θα πρέπει να αποφεύγεται όπου μας ενδιαφέρει η ακρίβεια και στη θέση του να χρησιμοποιείται η κλάση Java.math.BigDecimal. Για προσεγγιστική ισότητα χρησιμοποιήστε τη συνάρτηση

```
static boolean approximateEquals(double d1, double d2, double
epsilon) {
     return Math.abs(d2 - d1) < epsilon;
}
```

Ίσως να προσέξατε πως οι συναρτήσεις MIN_VALUE επιστρέφουν αρνητικές τιμές για τους ακέραιους τύπους και θετικές τιμές για τους float και double. Πράγματι, για τους πραγματικούς οι συναρτήσεις αυτές επιστρέφουν την πλησιέστερη τιμή στο 0. Αν χρειάζεστε την ελάχιστη αρνητική τιμή ενός πραγματικού τύπου, χρησιμοποιήστε –Float.MAX_VALUE και –Double.MAX_Value, αντίστοιχα.

boolean Οι μεταβλητές του τύπου boolean λαμβάνουν αυστηρά 2 δυνατές τιμές, true ή false. Ένα δυαδικό ψηφίο αρκεί για την αποθήκευση των τιμών αυτού του τύπου. Ωστόσο, οι προδιαγραφές της Java δεν καθορίζουν το μέγεθος του τύπου. Επομένως, το μέγεθος του τύπου boolean εξαρτάται από τη συγκεκριμένη υλοποίηση. Σε κάποιες γλώσσες είναι δυνατή η μετατροπή μεταξύ boolean και ακέραιων τύπων, στην Java δεν είναι επιτρεπτή τέτοια μετατροπή τύπου [1].
Στον κώδικα 3.7, παρουσιάζουμε παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου boolean:

```
static void booleans() { //Κώδικας 3.7
    boolean b1 = true, b2 = false;
    System.out.println("b1 equals to " + b1);
    System.out.println("b2 equals to " + b2);
```

```
    }
```

***Κώδικας 3.7**: Δήλωση και αρχικοποίηση boolean μεταβλητών*

Ακολουθεί η έξοδος του κώδικα 3.7.

```
b1 equals to true
b2 equals to false
```

char Ο τύπος char χρησιμοποιείται για την αποθήκευση χαρακτήρων. Υποστηρίζει το σύνολο Unicode, δηλαδή τους χαρακτήρες από τις περισσότερες γλώσσες συν μια ποικιλία χαρακτήρων που δεν συνιστούν γράμματα αλφάβητου και διάφορους χαρακτήρες ελέγχου (control characters) που είναι συνήθως μη εκτυπώσιμοι, όπως για παράδειγμα, ο χαρακτήρας που σηματοδοτεί το τέλος της τρέχουσας γραμμής (End Of Line). Στον κώδικα 3.8 δίνουμε παράδειγμα χρήσης μεταβλητών του τύπου char.

```
1 static void charDemo() { //Κώδικας 3.8
2         char cMin = Character.MIN_VALUE, cMax = Character.MAX_VALUE, cLit = 'A',
c = '\u0043';
3         System.out.println(cMin + " " + (int) cMin);
4         System.out.println(cMax + " " + (int) cMax);
5         System.out.println(c + " " + (int) c);
6         System.out.println(cLit + " " + (int) cLit);
7         System.out.println((char) (cLit + 1));
8     }
```

***Κώδικας 3.8**: Παράδειγμα χρήσης του τύπου char*

Ο κώδικας 3.8, έχει την ακόλουθη έξοδο

```
 0
□ 65535
C 67
A 65
B
```

Προσέξτε πως η κλάση που αντιστοιχεί στον τύπο char ονομάζεται Character. Σε κάθε μια από τις γραμμές 2-5 τυπώνουμε τον χαρακτήρα και στη συνέχεια μια τιμή που προκύπτει από τη μετατροπή του χαρακτήρα σε int. Η μετατροπή αυτή είναι ορθή γιατί οι χαρακτήρες κωδικοποιούνται στη μνήμη ως 16-bit μη προσημασμένοι ακέραιοι. Επομένως, ο char είναι και αυτός ένας ακέραιος τύπος. Πιο συγκεκριμένα, μια μεταβλητή τύπου char κρατάει την ακέραιη τιμή που συνιστά τον κωδικό του χαρακτήρα στο Unicode. Η έξοδος της γραμμής 2 φαίνεται ως ένα διάστημα ακολουθούμενο από το 0. Αυτό συμβαίνει γιατί ο cMin είναι ο NULL χαρακτήρας ο οποίος, όταν στέλνεται σε συσκευή εξόδου δεν κάνει τίποτα. Επομένως τυπώνεται το διάστημα που στέλνουμε στη γραμμή 2 και μετά ο κωδικός Unicode του NULL, δηλαδή το 0. Η γραμμή 3 τυπώνει ένα τετραγωνάκι ακολουθούμενο από τον κωδικό 65535 που στο δεκαεξαδικό σύστημα είναι το FFFF. Σημειώστε πως οι θέσεις του Unicode από FFF0 έως και FFFF χρησιμοποιούνται για ειδικές περιπτώσεις [3] που δεν ενδιαφέρουν στα πλαίσια αυτού του εγχειριδίου.

Προσέξτε πως η μεταβλητή c έχει αρχικοποιηθεί με τη σταθερά ‘\u0043’. Είναι εναλλακτικός τρόπος για έκφραση σταθερών τύπου char. Σύμφωνα με τον αυτόν τον τρόπο, ένας χαρακτήρας μπορεί να αναπαρασταθεί με \u ακολουθούμενο από τον δεκαεξαδικό κωδικό του. Θα δούμε περισσότερα σε σχέση με αυτό, αμέσως παρακάτω, στους χαρακτήρες διαφυγής. Η γραμμή 5 τυπώνει τον χαρακτήρα ‘A’ με τον κωδικό του. Στη γραμμή 6, προσθέτουμε το cLit που περιέχει το A με το 1. Η πρόσθεση αυτή είναι δυνατή, γιατί όπως εξηγήσαμε, στη μνήμη ένας char διατηρεί μια ακέραια τιμή που στη συγκεκριμένη περίπτωση είναι η τιμή 65. Επομένως, η τιμή της έκφραση cLit+1 είναι ίση με 66. Όταν το 66 μετατρέπεται σε char, στην ίδια γραμμή, τότε το αποτέλεσμα είναι ο χαρακτήρας B.

3.3 Ακολουθίες ελέγχου (Control Sequences)

Ο χαρακτήρας \ (backslash) ακολουθούμενος από έναν ειδικό χαρακτήρα ονομάζεται ακολουθία ελέγχου (control sequence) ή χαρακτήρας διαφυγής (escape character).
Στον πίνακα 3.1 παρουσιάζονται οι ακολουθίες ελέγχου της Java.

\t	tab	Αφήνει ένα διάστημα
\b	backspace	Μεταφέρει την επόμενη έξοδο έναν χαρακτήρα πίσω
\n	newline	Μεταφέρει την επόμενη έξοδο στην επόμενη γραμμή
\r	carriage return	Μεταφέρει την επόμενη έξοδο στην αρχή της τρέχουσας γραμμής
\f	form feed	Αλλαγή σελίδας στον εκτυπωτή
\'	single quote	Ο ειδικός χαρακτήρας ', λαμβάνεται ως κανονικός
\"	double quote	Ο ειδικός χαρακτήρας ", λαμβάνεται ως κανονικός
\\	backslash	Ο ειδικός χαρακτήρας \, λαμβάνεται ως κανονικός
\u	unicode character	Ακολουθούμενο από αριθμητική σταθερά αναπαριστά τον αντίστοιχο Unicode χαρακτήρα

Πίνακας 3.1 *Ακολουθίες Ελέγχου*

Ακολουθεί παράδειγμα χρήσης των ακολουθιών ελέγχου:

```
9      static void controlSequences() { //Κώδικας 3.9
10          System.out.println("slash_t" + '\t' + "second ");
11          System.out.println("slash_b" + '\b' + "second ");
12          System.out.println("slash_n" + '\n' + "second ");
13          System.out.println("slash_r" + '\r' + "second ");
14          System.out.println("slash_singlequote" + '\'' + "second ");
15          System.out.println("slash_doublequote" + '\"' + "second ");
16          System.out.println("slash_backslash" + '\\' + "second ");
17          System.out.println("slash_unicode" + '\u0041' + "second ");
18      }
```

Κώδικας 3.9*: Παραδείγματα χρήσης ακολουθιών ελέγχου*

Η έξοδος του κώδικα έχει ως εξής:

```
slash_t         second
slash_second
slash_n
second
second
slash_singlequote'second
slash_doublequote"second
slash_backslash\second
slash_unicodeAsecond
```

Η γραμμή 10 του κώδικα 3.9, τυπώνει “slash_t”, μετά αφήνει ένα tab και τέλος τυπώνει τη σειρά “second”.
Στη γραμμή 11, ο χαρακτήρας ‘\b’ ΄φέρνει τη σειρά “second” έναν χαρακτήρα πιο πίσω με αποτέλεσμα την εξαφάνιση του b από την “slash_b”.
Στη γραμμή 12, ο χαρακτήρας newline έχει ως αποτέλεσμα να εκτυπωθεί η “slash_n” και η “second” να εκτυπωθεί στην επόμενη γραμμή.
Στη γραμμή 13, το carriage return φέρνει την έξοδο στην αρχή της γραμμής με αποτέλεσμα να εξαφανίζεται η σειρά “slash_r”.
Η γραμμή 14 τυπώνει ένα quote ανάμεσα στην “slash_singlequote” και στην “second”.
Η γραμμή 15 τυπώνει ένα double quote ανάμεσα στην “slash_doublequote” και στην “second”.
Η γραμμή 16 τυπώνει ένα backslash ανάμεσα στην “slash_backslash” και στην “second”.
Η γραμμή 17 τυπώνει τον χαρακτήρα ‘A’ ανάμεσα στην “slash_unicode” και στην “second”.

3.4 Αναγνωριστικά

Είναι προφανές από την μέχρι τώρα παρουσίαση πως κάθε τοπική μεταβλητή λαμβάνει ένα όνομα που αποτελεί το αναγνωριστικό (identifier) της. Η σύνθεση των αναγνωριστικών υπακούει σε κανόνες. Πιο συγκεκριμένα, ο πρώτος χαρακτήρας υποχρεωτικά πρέπει να είναι ένας χαρακτήρας του Αγγλικού αλφάβητου, ο χαρακτήρας $ (dollar) ή ο χαρακτήρας _ (underscore). Στη συνέχεια, επιπλέον των προαναφερόμενων χαρακτήρων, μπορούν να χρησιμοποιηθούν και ψηφία.
Στον πίνακα 3.2, δίνονται παραδείγματα από έγκυρα και μη έγκυρα αναγνωριστικά

Έγκυρα Αναγνωριστικά	**Μη έγκυρα**
rVal	!color
costPerUnit	1start
event23	*profit
net_weight	start-up
$score	last/digit

Πίνακας 3.2 *Παραδείγματα έγκυρων και άκυρων αναγνωριστικών*

Εκτός από αυτούς τους κανόνες που είναι υποχρεωτικοί καθώς επιβάλλονται από τον μεταγλωττιστή, υπάρχει και μια σειρά από συμβάσεις που διέπουν την ονοματολογία των αναγνωριστικών των μεταβλητών. Οι συμβάσεις δεν επιβάλλονται από τον μεταγλωττιστή, έχουν ωστόσο καθολική αποδοχή και θα πρέπει να ακολουθούνται συστηματικά.

Από σύμβαση, τα αναγνωριστικά των μεταβλητών πρέπει να αρχίζουν με πεζό χαρακτήρα του αγγλικού αλφάβητου. Επομένως, οι χαρακτήρες dollar και underscore πρέπει να αποφεύγονται από την πρώτη θέση. Ειδικότερα ο χαρακτήρας dollar πρέπει να αποφεύγεται και από τις υπόλοιπες θέσεις. Θα πρέπει να χρησιμοποιούνται πλήρεις λέξεις ώστε ο κώδικας να είναι αναγνώσιμος και αυτοτεκμηριωμένος (self-documented). Για παράδειγμα, το αναγνωριστικό totalCost είναι πολύ σαφέστερο σε σχέση με τη σύντμηση tC. Η Java διαφοροποιεί μεταξύ πεζών και κεφαλαίων (case sensitive). Το αναγνωριστικό totalCost είναι διαφορετικό από το αναγνωριστικό totalcost. Αν ένα αναγνωριστικό μεταβλητής αποτελείται από μια λέξη, τότε όλα τα γράμματα στην ονομασία του πρέπει να είναι πεζοί χαρακτήρες. Αν όμως αποτελείται από περισσότερες από μία λέξεις, όλες οι επόμενες πρέπει να αρχίζουν από κεφαλαίο, π.χ. counter, customerWeight, first, firstOccurance.

Οι κανόνες και οι συμβάσεις για τα αναγνωριστικά των μεταβλητών ισχύουν και για τις παραμέτρους. Αντίθετα, όλοι οι χαρακτήρες στα αναγνωριστικά των σταθερών μεταβλητών πρέπει να είναι κεφαλαίοι.

3.5 Υπερχείλιση

Κατά την επιλογή ενός αριθμητικού τύπου, θα πρέπει να είμαστε σίγουροι πως η χωρητικότητά του επαρκεί για τα δεδομένα που θέλουμε να διαχειριστούμε. Σε περίπτωση που επιχειρήσουμε να εκχωρήσουμε τιμή έξω από τα όρια του τύπου, το αποτέλεσμα θα είναι να υπερχειλίσει (overflow) η μνήμη της μεταβλητής.
Στον κώδικα 3.10, παρουσιάζουμε ένα παράδειγμα υπερχείλισης:

```
static void overflow() { //Κώδικας 3.10
    int i = Integer.MAX_VALUE;
    i = i + 1;
    System.out.println(i);
    System.out.println(Integer.MIN_VALUE - 1);
}
```

Κώδικας 3.10*: Υπερχείλιση ακέραιων*

Η έξοδος της overflow είναι

```
-2147483648
2147483647
```

Η έξοδος αυτή είναι το αποτέλεσμα της υπερχείλισης. Καθώς τα αθροίσματα υπολογίζονται κατά τον χρόνο εκτέλεσης, ο μεταγλωττιστής δεν είναι σε θέση να μας προειδοποιήσει. Ως αποτέλεσμα έχουμε την εισαγωγή ενός λάθους χρόνου εκτέλεσης (run-time error) στο πρόγραμμά μας. Τα λάθη χρόνου εκτέλεσης είναι πολύ πιο σοβαρά από τα λάθη μεταγλώττισης ακριβώς γιατί δεν εντοπίζονται κατά τη μεταγλώττιση και ταξιδεύουν ως τον χρήστη του λογισμικού μας όπου μπορεί να προξενήσουν σημαντική ζημιά. Φανταστείτε τις πιθανές επιπτώσεις ενός τέτοιου λάθους σε ένα λογισμικό πλοήγησης αεροσκάφους.

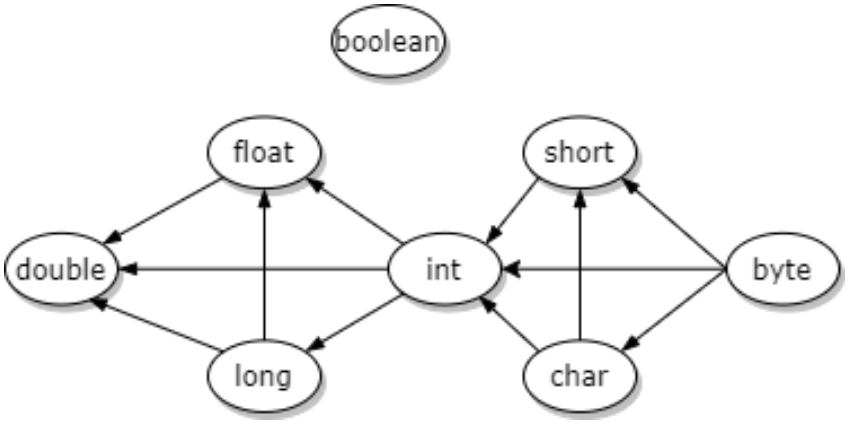
Αν παρόλα αυτά θέλετε να προσθέσετε δύο int και δεν είστε βέβαιοι ότι το άθροισμά τους είναι στα όρια του τύπου, μπορείτε να κάνετε μετατροπή τύπου τουλάχιστον στον ένα από τους δύο και στη συνέχεια να εκχωρήσετε το αποτέλεσμα σε έναν long. Στον κώδικα 3.11 δίνεται σχετικό παράδειγμα:

```
static void notOverflow() { //Κώδικας 3.11
    int i = Integer.MAX_VALUE;
    long k = (long) i + 1;
    System.out.println(k);
}
```

Κώδικας 3.11*: Άθροισμα ακέραιων εκτός ορίων του τύπου*

Η μετατροπή του τύπου του ενός ορίσματος σε long έχει ως αποτέλεσμα ο τύπος του αθροίσματος να είναι long. Επομένως, αν επιχειρήσουμε να εκχωρήσουμε το αποτέλεσμα αυτής της έκφρασης σε int, ο μεταγλωττιστής θα μας αποτρέψει. Αν όμως, όπως στο παράδειγμα, εκχωρήσουμε το άθροισμα σε μεταβλητή τύπου long, τότε θα λάβουμε το σωστό αποτέλεσμα, δηλαδή στο παράδειγμα τον αριθμό 2147483648.

Στην Java, ο τύπος των εκφράσεων στις οποίες συμμετέχουν οι αριθμητικοί τύποι byte και short είναι int. Επομένως, όταν προσθέτουμε 2 μεταβλητές τύπου short ή δύο τύπου byte ή μια short με μια byte, δεν χρειάζεται να μετατρέψουμε κανέναν τύπο, καθώς το αποτέλεσμα είναι int. Αυτό σημαίνει πως το αποτέλεσμα δεν μπορεί παρά να εκχωρηθεί σε μεταβλητή συμβατού τύπου, μεγαλύτερης ή ίσης χωρητικότητας από τον int. Μπορούμε ωστόσο να εκχωρήσουμε μια μεταβλητή ενός τύπου σε μεταβλητή άλλου τύπου με την προϋπόθεση πως οι τύποι είναι συμβατοί μεταξύ τους και ο τύπος της μεταβλητής που τοποθετείται δεξιά από τον τελεστή εκχώρησης, =, έχει μικρότερη χωρητικότητα από τον τύπο της μεταβλητής που βρίσκεται αριστερά. Σε αυτές τις περιπτώσεις, η Java προχωράει σε αυτόματη μετατροπή τύπου (automatic type conversion). Στο σχήμα 2, απεικονίζονται οι αυτόματες μετατροπές μεταξύ των θεμελιωδών τύπων.



Σχήμα 3.2*: Αυτόματες μετατροπές μεταξύ θεμελιωδών τύπων*

Όπως φαίνεται στο σχήμα 3.2, σε μια μεταβλητή τύπου double μπορεί να εκχωρηθεί τιμή οποιουδήποτε αριθμητικού τύπου, σε float οποιουδήποτε πλην του double, σε long οποιουδήποτε εκτός από double και float, σε int μπορούν να εκχωρηθούν short, char και byte και φυσικά int, σε short μπορούν να εκχωρηθούν char, byte και short, σε char μπορούν να εκχωρηθούν byte και char και σε byte μόνο byte. Ο τύπος boolean δεν εκχωρείται παρά μόνο σε boolean και κανένας άλλος τύπος δεν μετατρέπεται σε boolean. Περισσότερες πληροφορίες για τις μετατροπές τύπων μπορείτε να βρείτε στις προδιαγραφές της Java [4], §5.1.2, 5.1.3, 5.1.4.

Η υπερχείλιση ισχύει και για τους πραγματικούς τύπους, float και double. Ωστόσο, η συμπεριφορά εδώ είναι διαφορετική. Για παράδειγμα, η έκφραση Double.MAX_VALUE+1d επιστρέφει Double.MAX_VALUE. Μάλιστα, το ίδιο αποτέλεσμα έχουμε αν προσθέσουμε στον Double.MAX_VALUE και μεγαλύτερες τιμές. Μόνο αν προσθέσουμε τιμή μεγαλύτερη ή ίση του 2970, το αποτέλεσμα της έκφρασης θα γίνει Infinity. Επίσης,

η έκφραση Double.MIN_VALUE-1 επιστρέφει -1. Ο Double.MIN_VALUE είναι ένας πάρα πολύ μικρός αριθμός, τόσο μικρός ώστε μπορούμε να θεωρήσουμε ότι προσεγγίζει το μηδέν. Επομένως, αν από το 0 αφαιρέσουμε το 1 θα λάβουμε -1. Αντίστοιχα, αν διαιρέσουμε το Double.MIN_VALUE με το 2 θα λάβουμε ως αποτέλεσμα 0. Η περίεργη εκ πρώτης όψεως συμπεριφορά των πραγματικών μεταβλητών οφείλεται στον τρόπο αναπαράστασής τους. Προσέξτε πως μεταξύ δύο διαδοχικών πραγματικών αριθμών, x1 και x2, που μπορούν να αναπαρασταθούν με την κωδικοποίηση της IEEE, μεσολαβεί άπειρο πλήθος πραγματικών τιμών. Όλες αυτές οι τιμές κωδικοποιούνται σαν x1 ή σαν x2. Ο προσεγγιστικός χαρακτήρας των πραγματικών αριθμών δεν οφείλεται ειδικά στην κωδικοποίηση IEEE. Ένα διαφορετικό σύστημα κωδικοποίησης θα μπορούσε να παρέχει μεγαλύτερη ακρίβεια αλλά ο προσεγγιστικός χαρακτήρας παραμένει. Εξάλλου μεταξύ 2 οποιωνδήποτε πραγματικών αριθμών, μεσολαβεί πάντα άπειρο πλήθος πραγματικών τιμών.

3.6 Αριθμητικές σταθερές

Για την έκφραση των κυριολεκτικών αριθμητικών σταθερών μπορούν να χρησιμοποιηθούν, πέραν του δεκαδικού συστήματος και το δυαδικό, το οκταδικό και δεκαεξαδικό σύστημα αρίθμησης. Όταν μια αριθμητική σταθερά αρχίζει από 0b και ακολουθείται από δυαδικά ψηφία, τότε ερμηνεύεται στο δυαδικό σύστημα. Ο κώδικας

```
int i = 0b10, j = 0b11;
System.out.println(i + j);
```

έχει ως αποτέλεσμα να εκτυπωθεί 5, καθώς προσθέτει το 10 με βάση το 2 που είναι ίσο με το 2 στο δεκαδικό σύστημα, με το 11 του δυαδικού που είναι ίσο με το 3 του δεκαδικού.
Όταν μια αριθμητική σταθερά αρχίζει από 0 και ακολουθείται από τα ψηφία του οκταδικού συστήματος, ερμηνεύεται σαν οκταδικός αριθμός. Ο κώδικας

```
int i=010, j=011;
System.out.println(i+j);
```

έχει ως αποτέλεσμα να εκτυπωθεί 17, καθώς προσθέτει το 10 με βάση το 8 που είναι ίσο με το 8 στο δεκαδικό σύστημα με το 11 του οκταδικού που είναι ίσο με το 9 του δεκαδικού.
Όταν μια αριθμητική σταθερά αρχίζει από 0x και ακολουθείται από τα ψηφία του δεκαεξαδικού συστήματος, ερμηνεύεται σαν δεκαεξαδικός αριθμός. Ο κώδικας

```
int i=0x10, j=0x11;
System.out.println(i+j);
```

έχει ως αποτέλεσμα να εκτυπωθεί 33, καθώς προσθέτει το 10 με βάση το 16 που είναι ίσο με το 16 στο δεκαδικό σύστημα, με το 11 του δεκαεξαδικού που είναι ίσο με το 17 του δεκαδικού.

3.7 Αλφαριθμητικές Σειρές

Σε πολλές περιπτώσεις χρειάζεται να διαχειριστούμε δεδομένα όπως επωνυμίες και μηνύματα, δηλαδή σειρές από αλφαριθμητικούς χαρακτήρες. Για αυτές τις περιπτώσεις, η Java παρέχει τους τύπους String, StringBuffer και StringBuilder.

Το String δεν είναι θεμελιώδης τύπος στην Java, λόγω όμως της εκτεταμένης χρήσης του και της ανάγκης να αξιοποιείται σε παραδείγματα από τα πρώτα βήματα στη μελέτη της γλώσσας, παρουσιάζονται εδώ κάποιες αναγκαίες πληροφορίες. Ήδη στο πρώτο μας πρόγραμμα στην ενότητα 3, χρησιμοποιήσαμε σταθερές τύπου String. Ο κώδικας

```
System.out.println("Hello World!");
```

χρησιμοποιεί την αλφαριθμητική σταθερά "Hello World!" για να τυπώσει το κατάλληλο μήνυμα στην οθόνη.

Οι σταθερές τύπου String εσωκλείονται σε διπλά εισαγωγικά (double quotes), σε αντίθεση με τις σταθερές τύπου char που εσωκλείονται σε μονά εισαγωγικά. Μια πράξη που γίνεται πολύ συχνά ανάμεσα σε String είναι η σύνδεση των String (concatenation). Η σύνδεση των String γίνεται με τη βοήθεια του τελεστή +.

```
String s1 = "Hello";
String s2 = "World";
System.out.println(s1 + " " + s2);
```

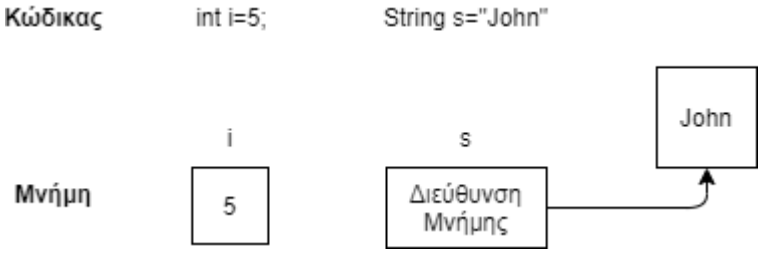
Κώδικας 3.11*: Παράδειγμα μεταβλητών τύπου String*

Στον κώδικα 3.11, στην πρώτη γραμμή, δηλώνουμε και αρχικοποιούμε τη μεταβλητή s1 τύπου String, στη δεύτερη γραμμή, δηλώνουμε και αρχικοποιούμε τη μεταβλητή s2. Τέλος, στην τρίτη γραμμή, συνδέουμε το s1 με μια σταθερά τύπου String που αποτελείται μόνο από τον χαρακτήρα διάστημα (space) και το αποτέλεσμα αυτής της σύνδεσης το συνδέουμε με το String s2. Ως αποτέλεσμα, τυπώνεται Hello World.

Προσέξτε πως η ονομασία του τύπου String ξεκινά με κεφαλαίο χαρακτήρα. Αυτό οφείλεται στο ότι ο τύπος δεν είναι θεμελιώδης, αλλά συνιστά μια κλάση και όπως έχουμε επισημάνει, από σύμβαση τα ονόματα των κλάσεων ξεκινούν με κεφαλαίο.

3.8 Μεταβλητές τιμής και αναφοράς

Μια άλλη πολύ ουσιαστική διαφορά μεταξύ String και θεμελιωδών τύπων είναι πως οι μεταβλητές τύπου String είναι μεταβλητές αναφοράς (reference variables) σε αντίθεση με τις μεταβλητές των θεμελιωδών τύπων που είναι μεταβλητές τιμής (value variables). Όταν σε μία μεταβλητή π.χ. τύπου int εκχωρούμε μια τιμή, τότε στη μνήμη της μεταβλητής εκχωρείται αυτή καθαυτή η τιμή. Αντίθετα, όταν σε μεταβλητή τύπου String εκχωρούμε μια τιμή, τότε δεσμεύεται χώρος στη μνήμη δυναμικά, στον χώρο αυτόν αποθηκεύεται η τιμή ενώ στη μεταβλητή αποθηκεύεται η διεύθυνση της τιμής, όπως δείχνει το σχήμα 3.3.



Σχήμα 3.3 *Μεταβλητές τιμής και αναφοράς*

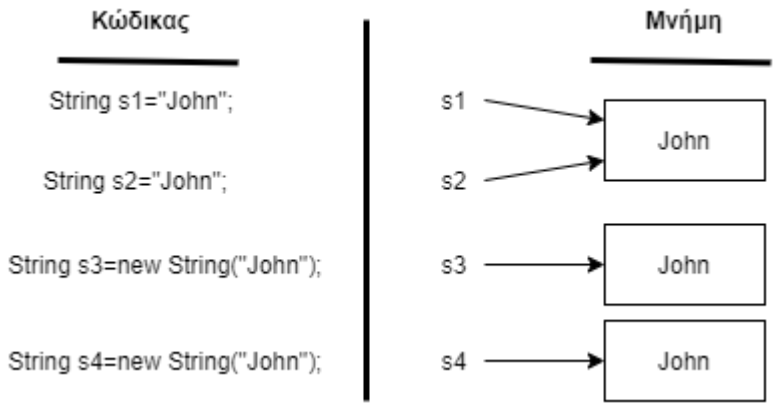
Στην τυπική περίπτωση, οι μεταβλητές αναφοράς δημιουργούνται με τη χρήση της λέξης κλειδί new. Για παράδειγμα, η δημιουργία ενός String γίνεται ως εξής:

```
String s=new String("John");
```

Ειδικά για τον τύπο String και για τις κλάσεις των θεμελιωδών τύπων παρέχεται η δυνατότητα δημιουργίας μεταβλητών χωρίς χρήση της λέξης new. Για παράδειγμα,

```
String s="John";
Integer i=125;
```

Όμως οι δύο τρόποι έχουν ουσιαστική διαφορά. Κάθε φορά που χρησιμοποιείται η new για τη δημιουργία μιας μεταβλητής δεσμεύεται νέος χώρος για την καταχώριση της τιμής της μεταβλητής. Όταν όμως η δημιουργία των μεταβλητών γίνεται χωρίς το new, τότε η ίδια τιμή αποθηκεύεται στην ίδια διεύθυνση μνήμης.



Σχήμα 3.4 *Δημιουργία μεταβλητών με την new*

Όπως δείχνει το σχήμα 3.4, οι μεταβλητές s1 και s2 δημιουργήθηκαν χωρίς χρήση της λέξης κλειδί new. Εφόσον έχουν την ίδια τιμή, κατανέμεται ο ίδιος χώρος στη μνήμη. Αντίθετα, οι μεταβλητές s3 και s4 δημιουργήθηκαν με χρήση της new. Σε αυτήν την περίπτωση κατανέμεται διαφορετικός χώρος στη μνήμη ακόμη και αν τα περιεχόμενα των μεταβλητών είναι ίδια. Επομένως, η s1 και η s2 περιέχουν την ίδια διεύθυνση μνήμης και άρα ισχύει s1==s2, δηλαδή οι μεταβλητές συγκρινόμενες με τον τελεστή ισότητας (ενότητα 4.1.4.1) θεωρούνται ίσες. Οι μεταβλητές s3 και s4 αναφέρονται στο ίδιο περιεχόμενο που είναι κατανεμημένο όμως σε διαφορετικές θέσεις στη μνήμη. Επομένως, στην s3 αποθηκεύεται διαφορετική διεύθυνση από ότι στην s4 και συνεπώς δεν ισχύει η ισότητα μεταξύ s3 και s4.
Αυτή η διαχείριση έχει ως στόχο τη βελτίωση της αποτελεσματικότητας του κώδικα και βασίζεται στο γεγονός ότι οι τιμές του τύπου String και των κλάσεων των θεμελιωδών τύπων είναι αμετάβλητες (immutable), δηλαδή από τη στιγμή που θα δημιουργηθεί μια τιμή ενός τέτοιου τύπου διασφαλίζεται πως δεν θα μεταβληθεί ως το τέλος του προγράμματος. Οι αμετάβλητοι τύποι εξυπηρετούν στον πολυνηματικό προγραμματισμό και χαρακτηρίζονται ως νηματικά ασφαλείς (thread safe).

3.8.1 Συλλέκτης απορριμμάτων

Εδώ όμως προκύπτει ένα άλλο θέμα. Το θέμα της απελευθέρωσης της μνήμης από τιμές μεταβλητών αναφοράς που δεν είναι χρήσιμες πλέον. Οι τοπικές μεταβλητές όπως είδαμε καταστρέφονται στο τέλος του μπλοκ στο οποίο δηλώθηκαν. Αυτό ισχύει είτε είναι μεταβλητές τιμής είτε μεταβλητές αναφοράς. Τι γίνεται όμως με τις τιμές των μεταβλητών αναφοράς που δεσμεύουν χώρο στη μνήμη δυναμικά; Είναι ένα θέμα που στην τυπική περίπτωση δεν απασχολεί ιδιαίτερα τον προγραμματιστή της Java καθώς η διαχείριση της δυναμικής μνήμης στην Java γίνεται από τον συλλέκτη απορριμμάτων (garbage collector). Ο συλλέκτης απορριμμάτων θεωρεί τις τιμές αναφοράς ως επιλέξιμες για αποδέσμευση μνήμης όταν δεν υπάρχει προς αυτές καμιά αναφορά μέσα στο πρόγραμμα.

```
static void garbageCollection() { //Κώδικας 3.4
    String s = new String("John");
    s = new String("Mary");
    System.out.println(s);
}
```

Κώδικας 3.4 Επιλέξιμες τιμές αναφοράς για αποδέσμευση μνήμης

Στην τελευταία γραμμή της garbageCollection στον κώδικα 3.4, το String John είναι ήδη επιλέξιμο προς αποδέσμευση. Η μόνη αναφορά στο John είναι η s έτσι όπως δημιουργείται στην πρώτη γραμμή. Στη δεύτερη γραμμή όμως, η αναφορά s ανακατευθύνεται σε άλλη τιμή. Έτσι το String John μένει χωρίς καμία αναφορά, δηλαδή είναι αδύνατο να το προσπελάσουμε μέσα από το πρόγραμμά μας καθώς το πρόγραμμά μας δεν γνωρίζει τη διεύθυνσή του. Με το τέλος της εκτέλεσης της garbageCollection, η τοπική μεταβλητή s καταστρέφεται, οπότε μένει και το String Mary χωρίς αναφορά και άρα καθίσταται και αυτό επιλέξιμο προς αποδέσμευση.

Το πότε ακριβώς θα πραγματοποιηθεί η αποδέσμευση της μνήμης εξαρτάται από τον συλλέκτη απορριμμάτων ο οποίος έχει δικούς του αλγόριθμους με στόχο την αποδοτική διαχείριση της μνήμης.

3.9 Ασκήσεις

1. Περνάει με επιτυχία μεταγλώττιση η x1;

```
static void x1() {
     int k = 1, K = 2;
     System.out.println(k + " " + K);
}
```

Παραβιάζει κάποια από τις συμβάσεις ονοματολογίας; Εξηγήστε την απάντησή σας.

2. Περνάει με επιτυχία μεταγλώττιση η x2; Εξηγήστε την απάντησή σας.

```
static void x2() {
     int k = 1, j;
     j = j + 1;
     System.out.println(k + " " + j);
}
```

3. Περνάει με επιτυχία μεταγλώττιση η x3; Εξηγήστε την απάντησή σας.

```
static void x3() {
     int k = 1;
     final int j = k;
     j = j + 1;
     System.out.println(k + " " + j);
}
```

4. Περνάει με επιτυχία μεταγλώττιση η x4; Εξηγήστε την απάντησή σας.

```
static void x4() {
     int k = 1;
     {
          k = 2;
     }
     System.out.println(k);
}
```

5. Περνάει με επιτυχία μεταγλώττιση η x5; Εξηγήστε την απάντησή σας.

```
static void x5() {
     int k = 1;
     {
          int k = 2;
     }
     System.out.println(k);
}
```

6. Ποια είναι η έξοδος της x6;

```
static void x6() {
     int k = 010;
     System.out.println(k);
}
```

7. Ποια είναι η έξοδος της x7;

```
static void x7() {
     int k = 010, j = 10;
     System.out.println(k + j);
}
```

8. Ποια είναι η έξοδος της x8;

```
static void x8() {
     int k = 010, j = 0x10;
     System.out.println(k + j);
}
```

9. Ποια από τα παρακάτω ονόματα μεταβλητών παραβιάζουν τους κανόνες και ποια τις συμβάσεις ονοματολογίας;

```
int sum; char Start; final char END; boolean ExitLoop;
int finalsum; double 3oAthroisma; byte day; float float_10;
float _1_;
```

10. Υπάρχει διαφορά μεταξύ των κυριολεκτικών σταθερών 8 και 8.0;

Υποδείξεις: Προσπαθήστε να απαντήσετε χωρίς τη βοήθεια του μεταγλωττιστή. Χρησιμοποιήστε τον μεταγλωττιστή για να ελέγξετε τις απαντήσεις σας.

Βιβλιογραφία

[1] “Primitive Data Types (The Java™ Tutorials > Learning the Java Language > Language Basics).” https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html (accessed Sep. 14, 2021).

[2] “Java All-in-One For Dummies, 6th Edition | Wiley,” Wiley.com. https://www.wiley.com/en-bb/Java+All+in+One+For+Dummies%2C+6th+Edition-p-9781119680451 (accessed Sep. 14, 2021).

[3] “Unicode 13.0.0.” https://unicode.org/versions/Unicode13.0.0/ (accessed Sep. 14, 2021).

[4] “The Java® Language Specification, Chapter 5. Conversions and Contexts, §5.1.2, §5.1.3, §5.1.4.” https://docs.oracle.com/javase/specs/jls/se10/html/index.html (accessed Sep. 14, 2021).

Κεφάλαιο 4

Σύνοψη
Σε αυτήν την ενότητα παρουσιάζονται οι τελεστές της Java και αναλύεται η συμπεριφορά τους. Πιο συγκεκριμένα, αναλύονται ο τελεστής εκχώρησης (assignment), οι αριθμητικοί τελεστές (arithmetic operators), ο τελεστής σύνδεσης συμβολοσειρών (concatenation), οι μοναδιαίοι (unary) τελεστές, ο τελεστής ισότητας (equality), οι σχεσιακοί (relational) τελεστές, ο τριαδικός τελεστής συνθήκης (ternary), ο τελεστής ελέγχου τύπου (instanceof) καθώς και οι τελεστές ψηφίου προς ψηφίο (Bitwise) και κύλισης ψηφίου (shift operators). Επιπλέον, παρουσιάζονται βασικοί τρόποι διαμόρφωσης της εξόδου στην οθόνη και εισαγωγής δεδομένων από το πληκτρολόγιο.

Προαπαιτούμενη γνώση
Προαπαιτείται να έχετε εγκατεστημένο το NetBeans με Java έκδοσης μεγαλύτερης ή ίσης της 8 και να γνωρίζετε τους θεμελιώδεις τύπους της Java, να χρησιμοποιείτε τον τύπο String, τις τοπικές μεταβλητές και σταθερές.

Λέξεις κλειδιά
Τελεστής, Τελεστέος, Εκχώρηση.

4 Τελεστές και βασική Είσοδος και Έξοδος

Η Java ενσωματώνει μια σειρά από συναρτήσεις που χρησιμεύουν ώστε να μπορούμε να κάνουμε διάφορες πράξεις (operations). Κάθε πράξη περιλαμβάνει τον τελεστή (operator) και τους τελεστέους (operands). Για παράδειγμα, στην πράξη 5+3, τελεστής είναι το σύμβολο της πρόσθεσης, + και τελεστέοι τα 5 και 3. Προκειμένου να διευκολυνθεί η έκφραση στο πρόγραμμα τέτοιων πράξεων, η Java παρέχει ειδικό συντακτικό κλήσης των τελεστών έτσι ώστε οι ανάλογες εκφράσεις να μην ακολουθούν το τυπικό συντακτικό κλήσης συναρτήσεων αλλά να διατυπώνονται κατά τρόπο που ομοιάζει με την εκτεταμένη χρήση τους στα μαθηματικά και άλλους τομείς.

Ένα άλλο ζήτημα, αναγκαίο για τα προγράμματά μας, είναι η βασική είσοδος και έξοδος. Πρόκειται για τις διαδικασίες που επιτρέπουν την επικοινωνία και αλληλεπίδραση μεταξύ χρήστη και προγράμματος. Πώς δηλαδή θα πάρω δεδομένα από τον χρήστη και πώς θα παρουσιάσω σε αυτόν τις πληροφορίες που θέλω;

Σε αυτό το κεφάλαιο παρουσιάζουμε τους βασικούς τελεστές που υποστηρίζει η Java [1] ταξινομώντας τους σε κατηγορίες ανάλογα με τα χαρακτηριστικά τους καθώς και τις βασικές λειτουργίες εισόδου από το πληκτρολόγιο και εξόδου στην οθόνη.

4.1 Τελεστές

Οι τελεστές (operators) στην Java οργανώνονται ανάλογα με τον αριθμό τελεστέων (operands) που δέχονται. Έτσι, έχουμε τους μοναδιαίους (unary) τελεστές, δηλαδή αυτούς που δέχονται έναν τελεστέο, τους δυαδικούς (binary), δηλαδή αυτούς με δύο τελεστέους και έναν τριαδικό (ternary) τελεστή. Επίσης, οι τελεστές κατηγοριοποιούνται σε αριθμητικούς (arithmetic), υποθετικούς (conditional) ή λογικούς (logical), σχεσιακούς (relational), τελεστές ψηφίου (bitwise) και τους τελεστές εκχώρησης (assignment).
Σε μια αλγεβρική έκφραση είναι δυνατό να συνδυάζονται περισσότεροι από έναν τελεστές. Για παράδειγμα, στην αλγεβρική έκφραση 2+3x5, συνδυάζονται οι τελεστές πρόσθεσης και πολλαπλασιασμού. Όπως γνωρίζουμε από την άλγεβρα, ο πολλαπλασιασμός έχει προτεραιότητα σε σχέση με την πρόσθεση. Επομένως, 2+3x5=17. Παρόμοια και στις εκφράσεις (expressions) της Java είναι δυνατό να συνδυάζονται πολλοί τελεστές. Επομένως, για τον συνεπή υπολογισμό των εκφράσεων έχουν καθοριστεί προτεραιότητες και για τους τελεστές της Java. Στον πίνακα 4.1 παρουσιάζονται οι τελεστές διατεταγμένοι από υψηλότερη σε χαμηλότερη προτεραιότητα. Τελεστές στην ίδια γραμμή του πίνακα 4.1 έχουν ίση προτεραιότητα.

Τύπος	**Λειτουργία**	**Προτεραιότητα**
Μοναδιαίος	Μεταθεματική αύξηση/μείωση	++, --
	Προθεματική αύξηση/μείωση, συμπλήρωμα, λογικό NOT	++, --, ~, !

Αριθμητικοί	Πολλαπλασιασμός	*, /, %
	Πρόσθεση	+, -
Διολίσθησης	Διολίσθηση	<<, >>, >>>
Σχεσιακοί	Σύγκριση	<, >, <=, >=, instanceof
	Έλεγχος ισότητας	==, !=
Ψηφίου	AND ψηφίο προς ψηφίο	&
	XOR ψηφίο προς ψηφίο	^
	OR ψηφίο προς ψηφίο	|
Λογικοί	Λογικό AND	&&
	Λογικό OR	||
Τριαδικός		? :
Εκχώρησης	Εκχώρησης	=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=

Πίνακας 4.1*: Προτεραιότητα τελεστών*

Ας υποθέσουμε την αλγεβρική έκφραση x=5+3-2. Η πρόσθεση και η αφαίρεση έχουν ίδια προτεραιότητα μεταξύ τους και ψηλότερη από την εκχώρηση. Επομένως, πρώτα θα υπολογιστεί η έκφραση 5+3-2 και το αποτέλεσμά της θα εκχωρηθεί στη μεταβλητή x. Πώς όμως θα αξιολογηθεί το τμήμα 5+3-2; Ο κανόνας εδώ είναι πως η αξιολόγηση γίνεται από αριστερά προς τα δεξιά, δηλαδή γίνεται πρώτα η πρόσθεση και το αποτέλεσμά της συνιστά τελεστέο για την αφαίρεση. Το ίδιο ισχύει για τις περισσότερες εκφράσεις της Java. Οι υπολογισμοί γίνονται από αριστερά προς τα δεξιά για όλους τους τελεστές πλην της εκχώρησης όπου η αξιολόγηση γίνεται από δεξιά προς τα αριστερά.

4.1.1 Ο τελεστής εκχώρησης

Η εκχώρηση είναι δυαδική πράξη και ο βασικός τελεστής της συμβολίζεται με τον χαρακτήρα '='. Στη γραμμή 2 του κώδικα 4.1 δηλώνονται τρεις ακέραιες μεταβλητές και αρχικοποιούνται οι δύο από αυτές. Στη γραμμή 3, εκχωρείται η τιμή 8 στη μεταβλητή i. Συνηθίζεται η πρώτη εκχώρηση σε μεταβλητή να αποκαλείται αρχικοποίηση (initialization). Η έκφραση (expression) στη γραμμή 3, επομένως, μπορούμε να πούμε πως αρχικοποιεί την i. Η αξιολόγηση γίνεται από τη δεξιά προς την αριστερή πλευρά της έκφρασης. Αυτό σημαίνει πως πρώτα θα υπολογιστεί ότι υπάρχει στα δεξιά του τελεστή εκχώρησης και στη συνέχεια η τιμή που θα προκύψει θα εκχωρηθεί στην i. Βέβαια, στη συγκεκριμένη περίπτωση, δεν απαιτούνται ιδιαίτεροι υπολογισμοί δεδομένου ότι στα δεξιά του τελεστή υπάρχει μια αριθμητική σταθερά. Θα μπορούσε όμως να υπάρχει και μια έκφραση που το αποτέλεσμα της αξιολόγησής της θα ήταν ακέραιος. Στη συνέχεια αυτής της ενότητας θα δούμε αρκετά τέτοια παραδείγματα.

```
1      static void assignmentDemo() { //Κώδικας 4.1
2          int i, j = 5, k = 9;
3          i = 8;
4          System.out.println(j = i);
5          j = k = i;
6          System.out.println(i + " " + j + " " + k);
7      }
```

Κώδικας 4.1*: Ο τελεστής εκχώρησης*

Εκείνο όμως που έχει ιδιαίτερο ενδιαφέρον είναι πως η εκχώρηση επιστρέφει στο περιβάλλον κλήσης της μια τιμή και συγκεκριμένα την τιμή της i μετά την εκχώρηση. Αυτή η τιμή δεν αξιοποιείται στη γραμμή 3. Αξιοποιείται όμως στη γραμμή 4 που τυπώνει την έκφραση j=i. Αν τρέξετε τον κώδικα 4.1, θα διαπιστώσετε πως η τιμή που τυπώνεται είναι 8. Η εκχώρηση, αφού αξιολόγησε την τιμή του i, την εκχώρησε στην j και επέστρεψε στην println την τιμή του j μετά την εκχώρηση. Σε αυτό το χαρακτηριστικό βασίζεται και η υποστήριξη της πολλαπλής εκχώρησης (γραμμή 5).

Όπως φαίνεται στον πίνακα 4.1, είναι διαθέσιμοι και άλλοι τελεστές εκχώρησης. Η παρουσίασή τους γίνεται πιο κάτω σε αυτήν την ενότητα.